Guide démo live — Demo 2 (parcours utilisateur MaaS)

Public : client / décideur / audience non technique **Durée cible :** 10–12 minutes **Pré-requis :** Demo 1 déjà jouée (ou au moins le modèle `demo-simulator` publié en MaaS) et `make demo2-verify` **vert** avant d'entrer en salle.

Règle d'or (différente de la Demo 1) : Demo 1 se joue 100 % console, zéro terminal. **Demo 2 mélange volontairement les deux** : le dashboard pour tout ce qu'un vrai utilisateur fait (clés API, abonnements), et un terminal pour les tests automatisés qui prouvent les codes HTTP réels (403, 429) sans attendre 15 clics devant le client. Annonce ce mélange dès le début, ce n'est pas un oubli de préparation.

Message unique à retenir :

> « Demo 1 a montré comment publier un modèle en MaaS. Demo 2 répond à la question suivante : une fois publié, qui a le droit de l'appeler, avec quelles limites, et qu'est-ce que ça donne vraiment — côté admin qui gouverne, et côté utilisateur final qui consomme ? Tout ce que vous allez voir est réel : un vrai utilisateur non-admin, une vraie connexion OIDC, de vraies réponses HTTP — y compris les deux ou trois endroits où la réalité ne colle pas exactement à la documentation officielle. »

Avant d'entrer en scène

1. `make demo2-verify` → doit afficher **"All checks passed"**.
2. Deux sessions navigateur prêtes (ou une seule + reconnexion) : une admin, une `maas-demo2-user`.
3. Un terminal ouvert dans le dépôt, prêt à lancer les commandes `make demo2-*`.
4. Les deux fichiers de clé déjà présents dans `secrets/demo2/` (sinon, prévoir 2 min de marge pour les recréer en live à l'étape 3).

Fil rouge (à dire une fois, 20 s)

```
Demo 1 : publier le modèle en MaaS
   │
   └─ Demo 2 : qui a le droit de l'appeler ?
        ├─ 1. Avant tout accès : l'utilisateur ne voit rien
        ├─ 2. L'admin accorde l'accès (déclaratif, GitOps)
        ├─ 3. L'utilisateur crée sa clé API et appelle le modèle
        ├─ 4. Ce qui se passe quand la gouvernance est en défaut (403 / 429)
        ├─ 5. Qui voit quoi comme usage (admin vs utilisateur)
        └─ 6. Révoquer / régénérer une clé
```

Parcours clics + terminal

Étape 0 — Avant tout accès, côté utilisateur (1–2 min)

1. Session `maas-demo2-user` → **Gen AI studio** → **API keys / Subscriptions**.
2. Montre : *« Request a subscription »*, *« No API keys »*, *« No subscriptions available »*.
3. Ouvre quand même **Create API key** : le sélecteur d'abonnement est **désactivé**, pas juste grisé.

Phrase type :

> « Voici un vrai utilisateur non-admin, aujourd'hui sans aucun accès configuré. Remarquez le texte exact — on l'a vérifié précisément parce qu'un client nous avait signalé un texte légèrement différent de la doc officielle, et on a préféré reproduire le comportement réel plutôt que de supposer que la doc avait raison. »

Étape 1 — L'admin accorde l'accès, en déclaratif (2 min) — moment clé n°1

Terminal :

```
make demo2-grant-access
```

Montre la sortie : validation `dry-run=server` contre les CRD réelles, puis les deux `MaaSSubscription` et le `MaaSAuthPolicy` qui s'appliquent, puis `MaaSModelRef` qui repasse à `Ready`.

Phrase type :

> « C'est le chemin GitOps : trois petits manifestes, nommés clairement — un abonnement qui donne accès à un groupe de démo avec une limite de débit définie, un second abonnement volontairement minuscule qu'on réutilisera pour le test de quota, et la politique d'autorisation qui fait le lien. Tout est scopé à ce groupe et ce modèle précis — ça ne peut pas accorder d'accès à autre chose sur le sandbox, et c'est idempotent : le rejouer ne change rien. »

(Mentionner rapidement, sans s'arrêter : *« Le dashboard propose aussi une case "Create a matching authorization policy" pour faire ça en un clic côté UI — on montre ici le chemin déclaratif, celui qu'on a pu vérifier intégralement de bout en bout sur ce sandbox. »*)

Étape 2 — L'utilisateur découvre le modèle et crée sa clé (2 min)

1. Retour session `maas-demo2-user` → recharge l'onglet Subscriptions.
2. Montre : l'abonnement apparaît, avec la limite configurée visible en développant la ligne.
3. **Create API key** → sélectionne l'abonnement → montre l'expiration max (90 jours) → crée → révèle la clé une fois.

Phrase type :

> « Cet utilisateur voit maintenant exactement un abonnement, avec la limite que l'admin vient de configurer. Créer une clé la scope à cet abonnement précis — et la valeur n'est montrée qu'une seule fois. Si on ferme cette fenêtre sans la copier, elle est perdue pour de bon ; il faudrait révoquer et recréer. C'est une propriété de sécurité volontaire, pas un bug — on va justement reproduire ce cycle complet plus tard. »

Étape 3 — Inférence réelle (1–2 min)

Terminal :

```
make demo2-test-inference
```

Montre : `GET /v1/models` → `200`, l'id complet du modèle. `POST /v1/chat/completions` → `200`, une vraie réponse et un objet d'usage de tokens.

Phrase type :

> « Appel OpenAI-compatible réel, contre le même modèle natif que la Demo 1 a déployé — avec la clé de l'utilisateur final, pas un token admin. Les chiffres d'usage que vous voyez sont de vrais

> comptages de tokens pour cette requête précise, pas un exemple préfabriqué. »

Étape 4 — Tests négatifs automatisés (1–2 min) — moment clé n°2

Terminal :

```
make demo2-test-negative
```

Pendant que ça tourne (~90 s), raconter : ça retire temporairement la politique d'autorisation → vrai `403` ; retire temporairement l'abonnement précis de cette clé → vrai `403` avec *"requested subscription not found"* dans le corps ; puis envoie une petite séquence de requêtes bornée contre un abonnement dédié à quota minuscule → vrai `429` une fois le quota épuisé. Tout est restauré automatiquement à la fin.

Phrase type :

> « Trois vrais scénarios négatifs, chacun déclenchant la vraie couche de gouvernance — pas des codes simulés. Remarquez le dernier : on a délibérément créé un deuxième abonnement minuscule et isolé juste pour ce test, pour ne jamais toucher au quota d'un autre utilisateur du sandbox. »

Étape 5 — Qui voit quoi comme usage (1–2 min)

Terminal :

```
make demo2-check-usage
```

Montre les compteurs Prometheus réels de Limitador — `authorized_calls` / `limited_calls` — qui correspondent exactement aux chiffres du test qui vient de tourner.

Phrase type :

> « Ce sont les vraies métriques de Kuadrant/Limitador, déjà câblées dans le monitoring de la plateforme — pas quelque chose qu'on a construit pour cette démo. Une limite honnête à souligner : ces compteurs sont par modèle, pas par utilisateur. S'il vous faut de la facturation ou un suivi par abonné, c'est un vrai manque aujourd'hui, pas une fonctionnalité qu'on cache. »

(Si le temps le permet, repasser une fois sur l'onglet Subscriptions de l'utilisateur : *« Et côté utilisateur, il voit sa limite configurée, mais jamais sa consommation réelle restante. Le seul signal qu'il reçoit, c'est un vrai 429 s'il dépasse. »*)

Étape 6 — Révoquer / régénérer (1–2 min)

1. Dashboard, menu kebab sur une clé → **Revoke** → taper le nom exact pour confirmer.
2. Montre : la clé disparaît des clés actives ; rejoue le même appel `GET /v1/models` avec cette clé maintenant morte → vrai rejet (ce sandbox renvoie **`403`**, pas le `401` décrit par la doc — à dire explicitement, c'est un vrai constat).
3. Crée une clé fraîche → vérifie qu'elle fonctionne immédiatement.

Phrase type :

> « La révocation ici est réelle et immédiate, confirmée en réutilisant vraiment la clé morte et en se faisant rejeter. Et pour être précis avec vous : notre test montre que cette version installée renvoie 403 ici, pas le 401 décrit par la documentation officielle — on vous donne le comportement réel qu'on a mesuré, pas celui de la doc. »

Aide-mémoire 1 page (2e écran / post-it)

```
0. maas-demo2-user → aucun accès (texte exact UI)
1. make demo2-grant-access → MaaSSubscription + MaaSAuthPolicy
2. Dashboard user → Create API key (révélée une fois)
3. make demo2-test-inference → 200 + usage tokens réels
4. make demo2-test-negative → 403 / 403 / 429 (tout restauré ensuite)
5. make demo2-check-usage → compteurs Limitador réels
6. Dashboard → Revoke (403, pas 401) → nouvelle clé → OK
```

Ce qu'il ne faut pas faire en live

À éviter	Pourquoi
Dire "401" pour une clé révoquée	Le vrai comportement mesuré est **403** — annoncer le mauvais code casse la crédibilité si le client le teste lui-même
Toucher au compte Keycloak `admin` partagé	C'est un compte partagé du sandbox, pas scopé à cette démo — jamais de mutation en live dessus
Promettre un suivi d'usage par utilisateur	Limitador agrège uniquement par modèle aujourd'hui — pas par clé/utilisateur
Montrer "AI asset endpoints" pour générer une clé temporaire	Ce chemin renvoie un vrai `403` même pour un propriétaire de projet — utiliser la page "API keys"
Laisser tourner `make demo2-test-negative` sans prévenir que ça va restaurer automatiquement	Le client peut croire que la démo a planté pendant les ~90 s

---

FAQ express

Q : Pourquoi mélanger terminal et dashboard alors que la Demo 1 restait 100 % console ? R : Demo 1 montre le geste admin "déployer + publier", entièrement pilotable en clics. Demo 2 prouve des codes HTTP réels (403, 429) qui seraient trop lents à démontrer clic par clic en live — le terminal ici sert à *prouver*, pas à cacher de la complexité.

Q : Est-ce que les 403/429 sont simulés pour la démo ? R : Non. Chaque test retire réellement un objet de gouvernance (`MaaSSubscription` ou `MaaSAuthPolicy`), envoie un vrai appel HTTP, montre le vrai code reçu, puis restaure l'état — vérifiable par `make demo2-verify` juste après.

Q : Et si un test ne se comporte pas comme prévu en live ? R : `make demo2-verify` permet de vérifier l'état avant de commencer. Si un test échoue, tout est conçu pour se restaurer seul (y compris sur Ctrl-C) — voir `docs/demo2/05-troubleshooting.md` pour le détail technique, hors écran client.

Q : Pourquoi l'utilisateur ne voit jamais sa consommation réelle ? R : Constat vérifié sur cette version : le dashboard utilisateur affiche la limite configurée, jamais la consommation ni le quota restant. Honnête à dire au client plutôt qu'à éluder.

Annexe (hors pitch) — secours technique

Détail complet (matrice de tests, découvertes réelles vs doc, troubleshooting, reproduction sur sandbox neuf) : `docs/demo2/` et `DEMO2-RESULTS.md` à la racine du dépôt.